A READER'S QUESTION, ANSWERED FROM THE EVIDENCE

Someone stole the bitcoin. Was php2shell finally used for real?

A working exploit

Proven

A real-world victim

Still none

A researcher won a public "steal my bitcoin" bounty by breaking into a **deliberately vulnerable** server with php2shell, and published how. Here is what that does, and does not, mean.

One minute

- Someone built a server **on purpose** to be vulnerable to php2shell, put ~$2,000 of bitcoin on it, and invited the internet to steal it.
- On 25 October a skilled researcher did, in ~19 hours, and wrote it up.
- That is **real proof the flaw gives full control of a machine** — the strongest public demonstration so far.
- But it is **a game with a consenting target, not a victim.** Nobody was harmed.
- So it does **not** change the standing answer: **no named organisation breached** through php2shell in the real world.

Three different things people are mixing up

A won bounty is proof the bug works — not proof a real organisation was a victim.

The flaw itself

REAL AND CONFIRMED

php2shell (CVE-2026-109618 / -109619) is a genuine, critical remote-code-execution flaw in PHP, fixed on 21 October.

Given an unpatched server, it can hand an attacker full control. That has never been in doubt.

The "Steal My Bitcoin" challenge

A GAME, WON

A server its owner built to be hacked, with bitcoin as bait and a public invitation to try.

A researcher won the ~$2,000 bounty on 25 October after ~19 hours. Consenting target, no victim.

A real-world breach

STILL NONE NAMED

No organisation has said it was broken into through php2shell, in its own words or a regulator's.

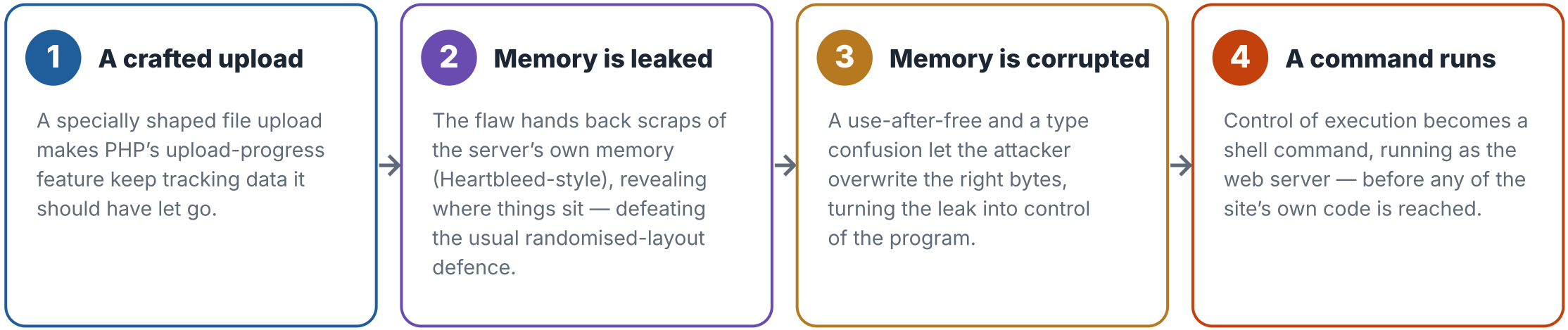
As of 12:14 UTC on 26 October, across 525 captured sources. This challenge does not change that.

Was it the same bug?

- **Yes.** It targeted **PHP 8.4.26** — one version below the 8.4.27 that carries the fix.
- It abused the same mechanism the PHP advisory names: the **session upload-progress** feature, the memory disclosure, and the memory-corruption flaws behind CVE-2026-109618 and CVE-2026-109619.
- **Not a new vulnerability.** It is the known php2shell family, demonstrated end to end on an unpatched build.

How the php2shell chain works, in four moves

The same mechanism the PHP advisory describes. Shown to explain, not to reproduce: no addresses, no payload.



Stopgap: turning the upload-progress feature off (session.upload_progress.enabled=0) blocks this path. Only updating to PHP 8.4.27 / 8.3.36 / 8.2.35 / 8.5.12 fixes the fl

A new attack vector? One wrinkle worth noting

- Not a new vulnerability — but a new **data point**.
- The usual advisories studied Apache **mod_php** and **PHP-FPM**. This challenge ran neither.
- It ran PHP's **built-in web server** (`php -S`) with worker processes — a mode the project says is **not for production**.
- The win shows that this built-in server, too, can be driven to full code execution. We have flagged it to add to the affected-systems table.

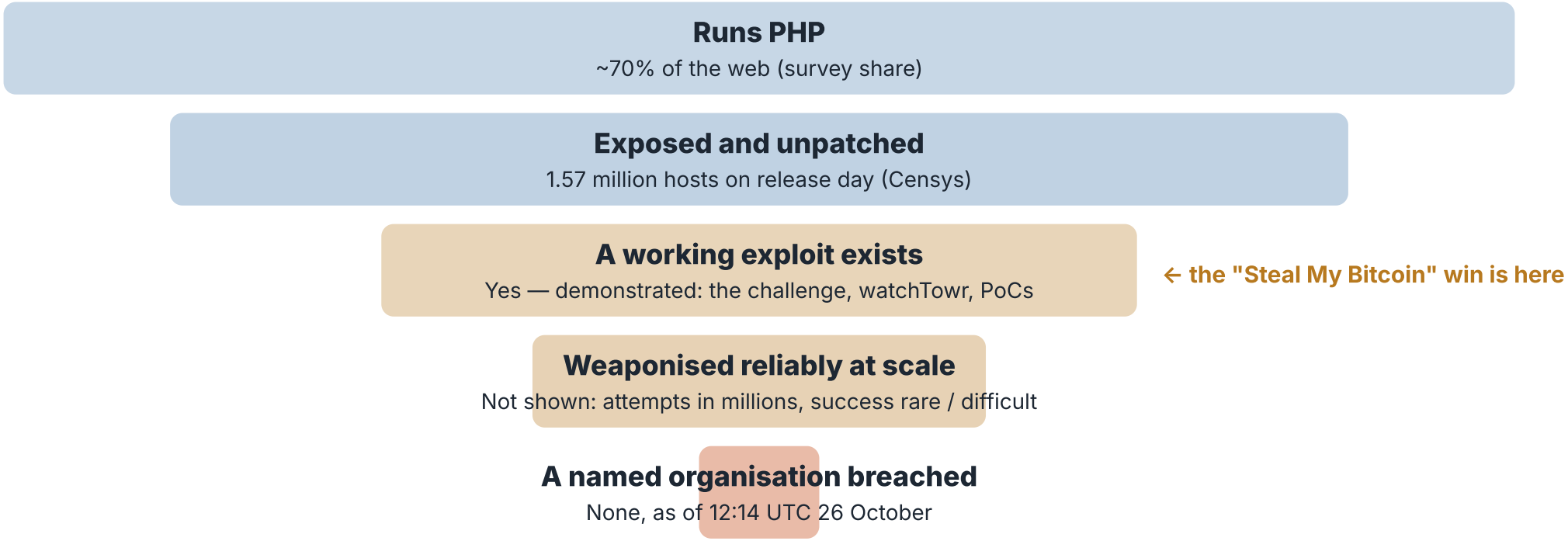
Why the class travels but this exploit does not

The vulnerability is general. This particular working exploit is bespoke to the challenge server.

Pinned to one PHP build **PHP 8.4.26 on Alpine, exact binary**	Memory-corruption offsets are specific to the compiled build; the write-up says a Debian stack would not match
An unusual server mode **PHP's built-in dev server (php -S) with multiple workers**	Not Apache mod_php or PHP-FPM — the modes the primary advisories studied; the built-in server is explicitly "not for production"
A race, not a certainty **~7 wins in 10 tries, in the author's own lab**	Timing-dependent; it fails much of the time even against the target it was tuned for
Re-tuned per target **offsets, layout and timing redone for each build**	Against a different production stack an attacker must redevelop the exploit, not re-run this one

From "runs PHP" to "was hacked": where php2shell actually sits

Each step is a smaller set than the one above. The challenge proved step 3; steps 4 and 5 are still not met.



What a defender should take from it

1. **Patch** to 8.5.12 / 8.4.27 / 8.3.36 / 8.2.35, or your distribution's backport. 8.4.26 is vulnerable.
2. As a stopgap, **turn off upload-progress** (`session.upload_progress.enabled=0`) — not a substitute for the update.
3. **Never run the built-in `php -S` server in production.**
4. **Assume memory disclosure beats ASLR**: a randomised layout is not a backstop here.
5. **Keep the proportion.** One consenting honeypot fell to a specialist in 19 hours. Real-world named victims: still zero.

> The full analysis, every claim sourced and fingerprinted:
> **https://php.oct.2026.newsroom.sgit.ai/articles/steal-my-bitcoin/**